

Beyond Miles Event Flow Project

Design Document

1. Introduction

This document serves as the initial design specification for the **Beyond Miles** project, a comprehensive travel booking web platform. It outlines the core requirements, data structures, and data flows within the system, providing a foundational understanding for development and implementation. The goal of Beyond Miles is to streamline package discovery, booking creation, traveler data capture, secure payment processing, and notifications offering a robust and user-friendly platform for customers, with strong administration capabilities for operators.

2. Functional Requirements

Functional requirements define the specific actions the system must perform. For Beyond Miles, these include:

2.1 User Management

- **User Registration:** The system shall allow new users to register with a unique email and password.
- **User Authentication:** The system shall allow registered users to log in securely.
- **Role-Based Access Control:** The system shall support roles (User, Admin) with distinct permissions using a role/permission mapping.
- **Profile Management:** Users shall be able to view and update their personal profiles, avatar, preferred currency, and preferences.
- **Email Verification:** The system shall support email verification before sensitive actions (e.g., booking, payment).

2.2 Package Management

- **Package Creation:** Administrators shall be able to create new travel packages specifying title, destination, duration, pricing, and status.

- **Package Editing:** Administrators shall be able to modify existing package details.
- **Publishing Controls:** Administrators shall be able to set package status (Active, Inactive, Draft) controlling visibility.
- **Package Deletion:** Administrators shall be able to delete or archive packages.
- **Package Listing:** Users shall be able to view a list of available packages with filtering and search capabilities.
- **Package Details View:** Users shall be able to view detailed information, including media, itinerary, inclusions, and exclusions.
- **Featured Packages:** The system shall allow marking packages as featured for the homepage.

2.3 Media, Itinerary, Inclusions/Exclusions & Reviews

- **Media Management:** Admins shall upload images/videos with ordering and alt text for accessibility.
- **Itinerary Management:** Admins shall manage day-wise itinerary items per package.
- **Inclusions/Exclusions:** Admins shall define what is included and excluded in each package.
- **Reviews:** Users shall be able to submit ratings and comments on packages; admins shall be able to reply and moderate.

2.4 Discounts & Promotions

- **Coupons:** Admins shall define coupons (percentage or flat) with validity windows, min order amounts, and caps.
- **Scoping:** Coupons shall be applicable globally or mapped to specific packages.
- **Validation:** The system shall validate coupons during checkout.

2.5 Booking & Travelers

- **Booking Creation:** Users shall create bookings by selecting a package, start date, and traveler count.

- **Pricing Computation:** The system shall compute subtotal, apply valid discounts, taxes, and determine the final total.
- **Traveler Data:** The system shall capture per-traveler details during booking.
- **Booking Status Management:** The system shall maintain booking statuses (Pending, Confirmed, Cancelled, Failed, Refunded).

2.6 Notifications

- **Automated Notifications:** The system shall send notifications for booking confirmations, payment statuses, and reminders via email/SMS.
- **Manual Notifications:** Admins shall be able to send custom notifications to users.
- **Template Rendering:** The system shall render templated notifications with dynamic payloads.
- **Status Tracking:** The system shall track notification status (queued, sent, failed).

2.7 Administration & Audit

- **RBAC:** The system shall enforce role-based authorization using roles, permissions, and mappings.
- **Audit Logs:** The system shall track admin actions (create/update/delete packages, booking changes) with before/after snapshots.

3. Data Schema Specification

This section outlines the proposed **relational** data schema for Beyond Miles. The following are the primary tables and their key fields:

3.1 Users

Represents registered users and authentication identity.

Column Name	Data Type	Constraints	Description
user_id	UUID	PK	Unique identifier for each user

name	VARCHAR(120)	NOT NULL	Full name
email	VARCHAR(190)	UNIQUE, NOT NULL	Email (used for login)
password_hash	VARCHAR(255)	NOT NULL	Hashed password
mobile	VARCHAR(20)	UNIQUE, NULL	Optional mobile
role	ENUM	DEFAULT 'USER'	USER, ADMIN
email_verified	BOOLEAN	DEFAULT FALSE	Email verification status
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time
updated_at	TIMESTAMP	DEFAULT NOW()	Last update time

Recommended Indexes: email (unique), mobile (unique).

3.2 User Profiles

Manages personal info separate from auth.

Column Name	Data Type	Constraints	Description
profile_id	UUID	PK	Unique profile identifier
user_id	UUID	FK → users.user_id, UNIQUE	Owner (1–1 relation)
avatar_url	VARCHAR(512)	NULL	Profile image
preferred_currency	VARCHAR(8)	NULL	e.g., INR, USD
preferences	JSON / JSONB	NULL	Travel interests, settings

Recommended Indexes: user_id (unique).

3.3 Travel Packages

Catalog items users browse and book.

Column Name	Data Type	Constraints	Description
package_id	UUID	PK	Unique package identifier
title	VARCHAR(180)	NOT NULL	Package title
slug	VARCHAR(200)	UNIQUE, NOT NULL	URL friendly unique slug
destination	VARCHAR(120)	NOT NULL	City/region/country
overview	TEXT	NULL	High-level description
duration_days	INT	NOT NULL	Total trip duration in days
base_price	NUMERIC(12,2)	NOT NULL	Base price per person/package
currency	VARCHAR(8)	NOT NULL	Currency code
is_featured	BOOLEAN	DEFAULT FALSE	Homepage featured flag
rating_avg	DECIMAL(3,2)	DEFAULT 0.00	Average rating 0–5
status	ENUM	DEFAULT 'Active'	Active, Inactive, Draft
created_at	TIMESTAMP	DEFAULT NOW()	Created time
updated_at	TIMESTAMP	DEFAULT NOW()	Updated time

Recommended Indexes: (destination, is_featured, status), status.

3.4 Package Media

Images/videos for package detail page.

Column Name	Data Type	Constraints	Description
media_id	UUID	PK	Unique media identifier
package_id	UUID	FK → travel_packages.package_id	Associated package
kind	ENUM	NOT NULL	Image, Video
url	VARCHAR(512)	NOT NULL	Media URL
alt_text	VARCHAR(255)	NULL	Accessibility text
sort_order	INT	DEFAULT 0	Display order
uploaded_by	UUID	FK → users.user_id	Uploader user
created_at	TIMESTAMP	DEFAULT NOW()	Uploaded at

Recommended Indexes: (package_id, sort_order).

3.5 Bookings

Booking header & financial summary.

Column Name	Data Type	Constraints	Description
booking_id	UUID	PK	Unique booking
booking_code	VARCHAR(20)	UNIQUE, NOT NULL	Human friendly code (e.g., BM-ABC123)
user_id	UUID	FK → users.user_id	Customer

package_id	UUID	FK → travel_packages.package_id	Booked package
start_date	DATE	NOT NULL	Trip start date
travelers_count	INT	NOT NULL	Number of travelers
price_subtotal	NUMERIC(12, 2)	NOT NULL	Subtotal before discounts/taxes
discount_applied	NUMERIC(12, 2)	DEFAULT 0.00	Discount total
tax_amount	NUMERIC(12, 2)	DEFAULT 0.00	Taxes
total_amount	NUMERIC(12, 2)	NOT NULL	Final amount
status	ENUM	DEFAULT 'Pending'	Pending, Confirmed, Cancelled, Failed, Refunded
created_at	TIMESTAMP	DEFAULT NOW()	Created time

Recommended Indexes: booking_code (unique), (user_id, created_at), (package_id, start_date).

3.6 Travelers

Per traveler information collected during booking.

Column Name	Data Type	Constraints	Description
traveler_id	UUID	PK	Unique traveler

booking_id	UUID	FK → bookings.booking_id	Parent booking
full_name	VARCHAR(120)	NOT NULL	Traveler name
age	INT	NULL	Age
gender	VARCHAR(20)	NULL	Optional
id_type	VARCHAR(40)	NULL	e.g., Passport, Aadhaar
id_number	VARCHAR(60)	NULL	ID number (should be protected/obfuscated)

Recommended Indexes: booking_id.

3.7 Payments

Razorpay orders/payments mapped to bookings.

Column Name	Data Type	Constraints	Description
payment_id	UUID	PK	Internal payment record
booking_id	UUID	FK → bookings.booking_id	Associated booking
gateway	VARCHAR(40)	NOT NULL	e.g., Razorpay
order_id	VARCHAR(100)	UNIQUE, NOT NULL	Razorpay Order ID
pg_payment_id	VARCHAR(100)	NULL	Razorpay Payment ID
signature	VARCHAR(255)	NULL	Signature for verification
amount	NUMERIC(12,2)	NOT NULL	Amount expected/paid
currency	VARCHAR(8)	NOT NULL	Currency

status	ENUM	DEFAULT 'Created'	Created, Paid, Failed, Refunded
method	VARCHAR(40)	NULL	card/upi/netbanking etc.
created_at	TIMESTAMP	DEFAULT NOW()	Created
captured_at	TIMESTAMP	NULL	When captured

Recommended Indexes: order_id (unique), (booking_id, status).

3.8 Admin Audit Logs

Track admin actions (add/update/delete packages, booking changes).

Column Name	Data Type	Constraints	Description
audit_id	UUID	PK	Unique audit record
actor_user_id	UUID	FK → users.user_id	Admin who performed action
action	VARCHAR(120)	NOT NULL	e.g., pkg.update
entity	VARCHAR(60)	NOT NULL	TravelPackage, Booking, Review
entity_id	UUID	NOT NULL	Target record id
before_state	JSON / JSONB	NULL	Snapshot before change
after_state	JSON / JSONB	NULL	Snapshot after change
created_at	TIMESTAMP	DEFAULT NOW()	When action happened

Recommended Indexes: (actor_user_id, created_at), (entity, entity_id).

4. Data Flow Specifications

This section details how information is captured, processed, and disseminated across the system, described in terms of external entities, processes, data stores, and the data flows between them.

4.1 User Registration and Authentication Data Flow

- **External Entity:** User (Customer, Administrator)
- **Process:** User Registration & Authentication
- **Data Store:** Users DB, User Profiles DB

Flow:

1. User provides **User Info** (name, email, password) to the Registration & Authentication process.
2. The process validates and stores user data in the **Users DB**, initializing email_verified = false.
3. The system sends a verification link; upon confirmation, it updates **Users DB** to set email_verified = true.
4. Upon successful login/verification, the process provides an **Auth Token** back to the User.

4.2 Package Creation and Management Data Flow

- **External Entity:** Administrator
- **Process:** Package Creation & Management
- **Data Store:** Packages DB, Media DB, Itinerary DB, Inclusions DB, Exclusions DB, Audit Logs DB

Flow:

1. Administrator provides **Package Details** (title, destination, duration, price, currency, status) to the Package Management process.
2. The process validates and stores/updates package data in the **Packages DB**, and media/itinerary/inclusions/exclusions in their respective stores.
3. The process logs **before/after** updates in **Admin Audit Logs DB**.
4. Setting status = Active makes the package visible in discovery APIs.

4.3 Booking and Payment Data Flow

- **External Entity:** User, Payment Gateway (Razorpay)

- **Processes:** Booking Creation, Discount Validation, Payment Processing, Notification System
- **Data Stores:** Bookings DB, Travelers DB, Discounts DB, Payments DB, Notifications DB

Flow:

1. User sends **Booking Request** (package ID, start date, travelers count, optional coupon) to the Booking process.
2. Booking process computes **Subtotal**, validates coupon from **Discounts DB**, applies caps/min amounts, and computes **Tax** to determine **Total**.
3. Booking is stored in **Bookings DB** with status Pending.
4. Payment process creates **Razorpay Order**, storing order_id in **Payments DB** (status = Created).
5. User completes payment; backend verifies **payment_id + signature** → on success: set **Payments = Paid, Bookings = Confirmed**.
6. The process triggers **Notifications** (EMAIL/SMS) via the Notification System.
7. On failure: set **Payments = Failed**; booking remains Pending or transitions to Failed based on business rules.
8. Refunds: update **Payments/Bookings** status accordingly and notify user.

4.4 Payment Webhooks Data Flow

- **External Entity:** Razorpay
- **Process:** Webhook Ingestion & Reconciliation
- **Data Stores:** Payment Webhooks DB, Payments DB, Bookings DB

Flow:

1. System receives webhook; stores raw payload with unique event_id in **Payment Webhooks DB**, processed = false.
2. Worker verifies HMAC signature and **idempotently** updates **Payments DB** (and **Bookings DB**, if needed).

3. Worker sets processed = true and may trigger **Notifications** depending on event type.

4.5 Notification System Data Flow

- **External Entity:** User (Customer, Administrator)
- **Process:** Notification System
- **Data Stores:** Notifications DB, Users DB, Bookings DB, Packages DB

Flow:

1. Notification System composes **Notification Content** (via templates) and sends to the User (EMAIL/SMS).
2. Notification System stores a **Notification Record** in **Notifications DB** with status and timestamp.

5. Conclusion

This design document provides a foundational blueprint for the **Beyond Miles** project, detailing its functional and non-functional requirements, proposed data schema, and the flow of information within the system. Adhering to these specifications will ensure the development of a robust, scalable, and user-friendly travel booking solution.